

# Advanced techniques in applied economics

## Modelling with graphs, Lectures 6-10

Piotr Zwiernik



**Universitat  
Pompeu Fabra**  
*Barcelona*

Statistics, Probability  
and Machine Learning  
Research Group



**Barcelona School of Economics**

February 18, 2026

## Part 4: Bayesian networks and causal discovery

## 4.1 DAGs and their models

## DAG terminology

**Directed acyclic Graph (DAG)** is a directed graph with no directed cycles.

Equivalently, we can order the vertices so that an arrow always goes from smaller to larger vertex.

**parents**  $\text{pa}(v)$ : nodes  $u$  such that  $u \rightarrow v$  in  $G$ ,

directed path  $u \rightarrow \cdots \rightarrow v$ ,

path  $u \leftrightarrow \cdots \leftrightarrow v$ ,

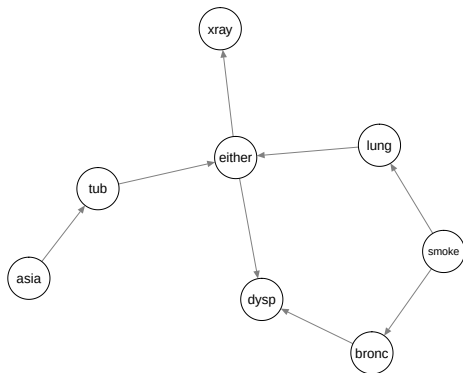
**descendants**  $\text{de}(v)$  all nodes  $u$  s.t. there is a directed path from  $v$  to  $u$ ,

**non-descendants**  $\text{nd}(v)$ : all the other nodes in  $V \setminus \{v\}$ .

**v-structure (immorality)**: **induced** subgraph of the form  $u \rightarrow w \leftarrow v$

## Example

DAG from the Chest Clinic Example Lauritzen & Spiegelhalter (1988). Shortness of breath (dyspnea) is the response variable.



Give an example of a v-structure. What are the parents of *either*? Is there a directed path from *bronc* to *xray*? Who are the descendants of *asia*? Who are the non-descendants?

## d-separation

A path  $\pi$  from vertex  $i$  to vertex  $j$  is **blocked** by  $S \subset V$  if  $\pi$  contains one of the following types of nodes:

- $v \in S$  such that the path looks locally like  $\rightarrow v \rightarrow$ ,  $\leftarrow v \leftarrow$ , or  $\leftarrow v \rightarrow$ .
- $v \notin S$  such that  $S \subseteq \text{nd}(v)$  and the path looks locally like  $\rightarrow v \leftarrow$ .

Two subsets  $A$  and  $B$  are said to be **d-separated** by  $S$  if all paths from  $A$  to  $B$  are blocked by  $S$ .

Package `bnlearn` has a function `dsep` to check d-separation:

```
> library(bnlearn)
> chestdag = model2network("[asia][tub|asia][smoke][lung|smoke][bronc|smoke][either|lung:tub][xray|
  either][dysp|bronc:either]")
> dsep(chestdag, "asia", "xray", c("lung","bronc"))
[1] FALSE
# the shortest path from asia to xray is not blocked by c("lung","bronc")
```

[Mention hierarchical layouts]

## DAG models/Bayesian networks

Let  $G$  be a directed acyclic graph (DAG).

We say that  $f(\mathbf{x})$  factorizes according to  $G$  if for all  $\mathbf{x} \in \mathcal{X}$

$$f(\mathbf{x}) = \prod_{v \in V} f(x_v | \mathbf{x}_{\text{pa}(v)}).$$

Factorization is equivalent to any of the following Markov properties:

(G)  $A \perp\!\!\!\perp B | S$  whenever  $S$  d-separates  $A$  and  $B$ .

(L)  $v \perp\!\!\!\perp \text{nd}(v) | \text{pa}(v)$  for every  $v \in G$ .

This results holds with no restrictions on the support of  $f$ .

We reiterate:  $i \perp\!\!\!\perp j | S$  if and only if  $S$  **d-separates**  $i$  and  $j$  in  $G$ .

# Belief propagation and inference

Belief propagation provides a computationally efficient way to compute various marginal and conditional distributions for a DAG model.

First we set up some distribution by specifying terms of the factorization.

```
library(gRain)
yn <- c("yes","no")
a <- cptable(~asia,values=c(1,99),levels=yn)
t.a <- cptable(~tub+asia,values=c(5,95,1,99),levels=yn)
s <- cptable(~smoke,values=c(5,5),levels=yn)
l.s <- cptable(~lung+smoke,values=c(1,9,1,99),levels=yn)
b.s <- cptable(~bronc+smoke,values=c(6,4,3,7),levels=yn)
e.lt <- cptable(~either+lung+tub,values=c(1,0,1,0,1,0,0,1),levels=yn)
x.e <- cptable(~xray+either,values=c(98,2,5,95),levels=yn)
d.be <- cptable(~dysp+bronc+either,values=c(9,1,7,3,8,2,1,9),levels=yn)
plist <- compileCPT(list(a,t.a,s,l.s,b.s,e.lt,x.e,d.be))
grn1 <- grain(plist) # Graphical Independence Network
grn1c <- compile(grn1)
grn1c <- propagate(grn1c) # at this point probabilistic queries can be asked
```



To compute various joint distributions and conditional distributions of a variable given a subset of variables we run.

```
> querygrain(grn1c,nodes=c("lung","bronc"),type="joint")
      bronc
lung yes no
yes 0.0315 0.0235
no 0.4185 0.5265

> querygrain(grn1c,nodes=c("dysp","asia"),type="conditional")
      dysp
asia yes no
yes 0.4501375 0.5498625
no 0.4358275 0.5641725
```

We can also see what happens when we fix values of some variables.

```
> grn1c.ev <- setFinding(grn1c,nodes=c("asia","smoke"),states=c("yes","yes"))
> querygrain(grn1c.ev,nodes=c("lung","bronc"),type="joint")
      bronc
lung yes no
yes 0.06 0.04
no 0.54 0.36
```

For examples see: [www.bnlearn.com/bnrepository/](http://www.bnlearn.com/bnrepository/)

## Learning distributions from data

In some special situations the DAG structure can be specified by an expert.

Given data we can then learn the underlying conditional probabilities.

```
> data("chestSim500", package='gRbase')
> simdagchest <- grain(as.graphNEL(chestdag), data=chestSim500)
> simdagchest <- compile(simdagchest, propagate=TRUE, smooth=.1)
```

Then inference can be made as explained earlier.

```
> querygrain(simdagchest, nodes=c("lung", "bronc"), type="joint")
      bronc
lung yes no
yes 0.02539682 0.02060318
no 0.42860318 0.52539682
```

Note that the estimated values of the probabilities are close to the truth.

## Learning the graph

Learning the underlying DAG from the data is a major problem.

Important links:

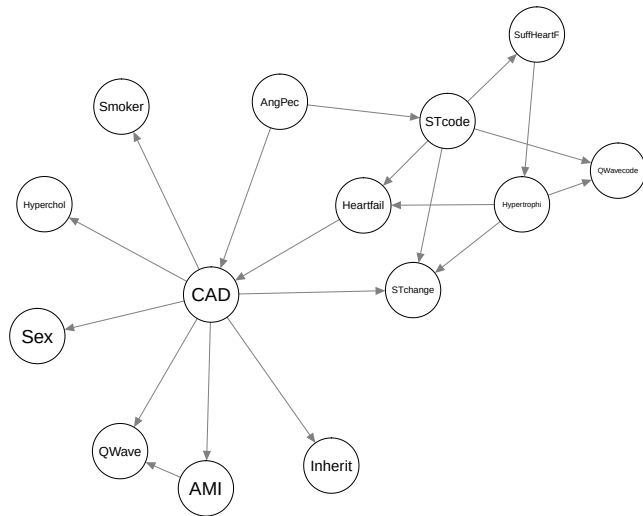
[annepetersen.shinyapps.io/causalDisco/](https://annepetersen.shinyapps.io/causalDisco/) – an overview of available methods to learn DAGs in R.

[www.bnlearn.com](http://www.bnlearn.com) – an overview of methods in bnlearn.

### Example: Coronary artery disease data

A cross classified table with observational data from a Danish heart clinic. The response variable is **coronary artery disease (CAD)**:

```
data(cad1, package='gRbase')
cad.bn <- hpc(cad1)
qgraph::qgraph(cad.bn)
# in case you want to exclude some edges
cad.bn.2 <- hc(cad1, blacklist=as.data.frame(matrix(c("CAD", "Sex"), 1, 2)))
```



It is important to incorporate prior knowledge. See <http://www.bnlearn.com/examples/whitelist/> for details.

# causalDisc



## Choose properties

- ☐ Assume faithfulness?
- ☐ Allow for unobserved variables?
- ☐ Allow for external restrictions on edge presence?
- ☐ Allow for external restrictions on edge absence?
- ☐ Allow for custom independence tests?
- ☐ Allow for external restrictions on edge directions?
- ☐ Score-based approach?
- ☐ Constraint-based approach?
- ☐ Assume linearity?
- ☐ Assume non-Gaussian errors?
- ☐ Estimation-based approach (complete identification)?
- ☐ Support missing information?
- ☐ Support numerical data?
- ☐ Support categorical data?
- ☐ Return DAG (all edges directed)?
- ☐ Support parallel computing?

Procedure information

[About the](#)

Select a procedure in the panel to the le

*If there are no available procedures for y application, please report it [here](#).*

## Faithfulness

If  $f$  factorizes with respect to the DAG  $G$  then

$$S \text{ d-separates } A \text{ and } B \text{ in } G \implies A \perp\!\!\!\perp B|S.$$

At least in principle,  $f$  may satisfy some additional conditional independence statements so the reverse implication may not hold.

$f$  is **faithful** to the DAG  $G$  if

$$S \text{ d-separates } A \text{ and } B \text{ in } G \iff A \perp\!\!\!\perp B|S.$$

This assumption is not entirely innocent:

Uhler, C., Raskutti, G., Bühlmann, P., & Yu, B. (2013). Geometry of the faithfulness assumption in causal inference. The Annals of Statistics.

# PC algorithm

Currently one of the most popular algorithms.

Stage one: Learn the skeleton.

Use the fact that  $u \not\sim v \iff \exists S \subseteq V \text{ s.t. } u \perp\!\!\!\perp v | S$ .

Starting with the complete graph, conditional independences with an increasing order are tested and edges are removed.

Stage two: Learn the edge directions.

Several algorithms are implemented in `pca1g`

# Application for CAD data

Load the data and learn the CP tables.

```
library(pcalg); library(gRbase)
# use again the CAD dataset. coding {0,1} and {0,1,2}
cad <- data.matrix(cad1)-1;
# 236 observations on 14 variables (one ternary, 13 binary)
```

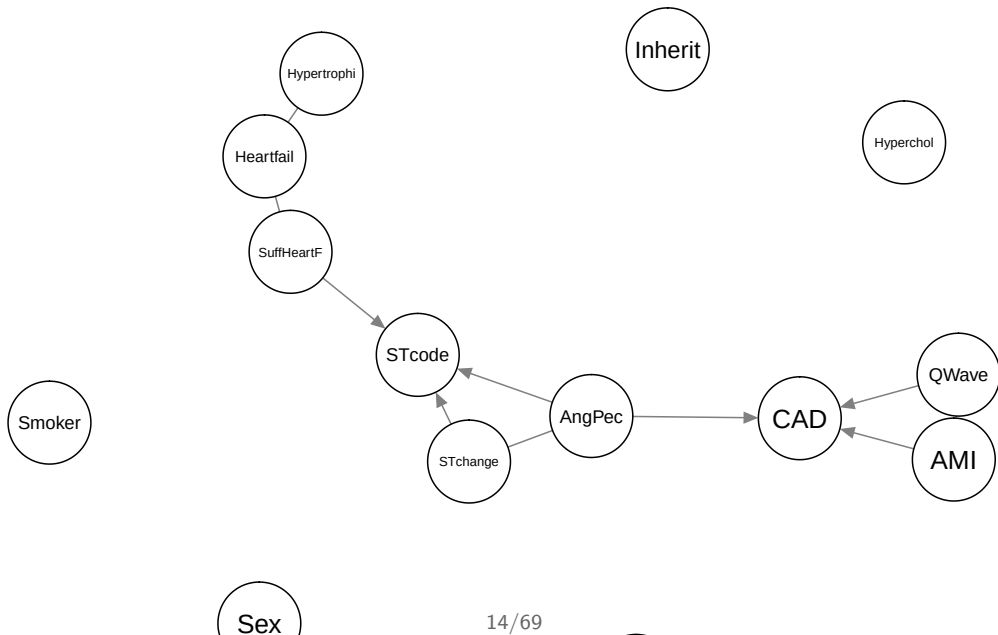
Run the PC algorithm with the implemented discrete independence test.

```
suffStat <- list(dm=cad,nlev=rep(2,14)+diag(14)[2,],adaptDF=TRUE)
cad.bn <- pc(suffStat,disCItest,alpha=0.1,labels=colnames(cad))
qgraph::qgraph(cad.bn)
```

The package has also implemented testing CI for binary distributions (function binCItest) and also any external function can be called here.

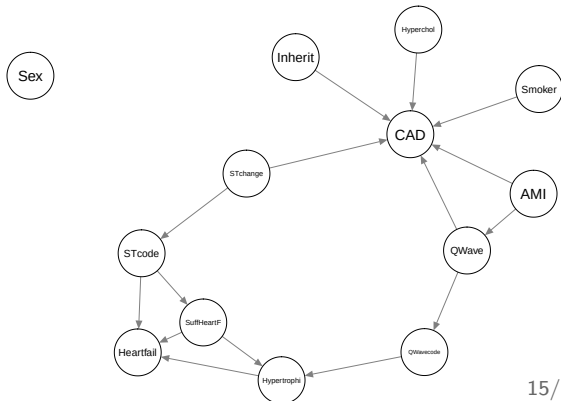
- may be important in case of small sample sizes as in the example above.





We use part of the CAD data to obtain a data set of binary variables.

```
# remove the ternary variable
cad2 <- data.matrix(cad1[,-2])-1;
# 236 observations on 13 binary variables
suffStat <- list(dm=cad2,adaptDF=TRUE)
# m.max=4 because otherwise the estimates unreliable
cad.bn <- pc(suffStat,binCITest,alpha=0.1,labels=colnames(cad2),m.max=4)
qgraph::qgraph(cad.bn)
```



# Markov equivalence

Only very rarely we can identify the underlying DAG from **observational data**.

DAGs can be learned only up to some equivalence class.

e.g.  $\overset{1}{\bullet} \rightarrow \overset{2}{\bullet}$ ,  $\overset{1}{\bullet} \leftarrow \overset{2}{\bullet}$ , and  $\overset{1}{\bullet} - \overset{2}{\bullet}$  all define the same model.

Two DAGs are **Markov equivalent** (define the same statistical model) if and only if they have the same skeleton and v-structures.

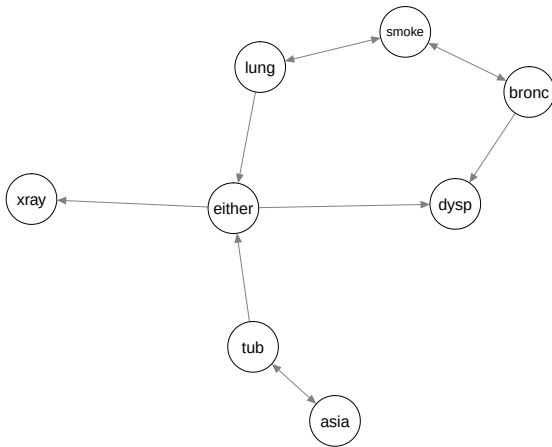
- **Essential graph** describes the Markov equivalence class.

In certain scenarios there is a way around this, which we will discuss:

- Use interventions (experimental economics?)
- Instead of a DAG, assume an underlying Structural Causal Model.
- Anchors and invariance. (not discussed here)

## Visualize the essential graph

```
library(ggm)
qgraph::qgraph(as(essentialGraph(amat(chestdag)), "graphNEL"))
```



## 4.2 Structural Causal Models

# Structural Causal Models

A structural causal model (SCM) consists of a collection of  $m$  structural assignments

$$X_i := f_i(\text{pa}_i, \epsilon_i) \quad \text{for } i = 1, \dots, m,$$

where  $\text{pa}_i \subset \{X_1, \dots, X_m\} \setminus \{X_i\}$  and all  $\epsilon_i$  are independent of each other.

The underlying graph  $G$  of an SCM is defined in the obvious way.

If  $G$  is a DAG, sampling data from SCMs is easy

Sample from the SCM:  $X_1 = X_3^2 + \epsilon_1$ ,  $X_2 = \sin(X_3) + \epsilon_2$ ,  $X_3 = \epsilon_3$ ,  $X_4 = X_2 + e^{X_3 + \epsilon_4}$ , with  $\epsilon_i$  having respectively normal, chi-squared, uniform, and normal distribution.

```
set.seed(1); n <- 100
X3 <- runif(n)-0.5
X1 <- X3*X3 + rnorm(n)
X2 <- sin(X1)+rnorm(n)^2
X4 <- X2+exp(X3+rnorm(n))
X <- cbind(X1,X2,X3,X4)
```

# Gaussian DAG models

## Linear Structural Equation models

Let  $\varepsilon_1, \dots, \varepsilon_m$  be independent variables. Consider a model

$$X_i = \sum_{j \in \text{pa}(i)} \lambda_{ij} X_j + \varepsilon_i.$$

The **Gaussian DAG models** are precisely the Linear Structural Equation models on DAGs, where  $\varepsilon_i$  are independent Gaussian variables.

The function `fitDag` in the **ggm** package fits Gaussian DAGs.

```
> library(ggm); data(carcass); S.carc <- cov(carcass)
> gdag1 <- DAG(LeanMeat~Meat13:Fat11:Fat12, Meat13~Meat11:Meat12, Fat12~Fat11, Fat13~Meat11:Meat12, Meat12~Meat11)
> qgraph::qgraph(as(gdag1,"graphNEL")) # to visualize the DAG
> fdag1 <- fitDag(gdag1, S.carc, nrow(carcass))
> fdag1$dev # the deviance is really large and the p-value is basically 0.
[1] 552.2726
```

# Interventions

Recall: Typically impossible to identify the DAG from observational data.

Things are better if we can observe the system under **intervention** or include the domain knowledge.

We consider only the simplest way to intervene by fixing a value of a variable.

Consider the example on Slide 18 with  $X_2 \leftarrow 1$

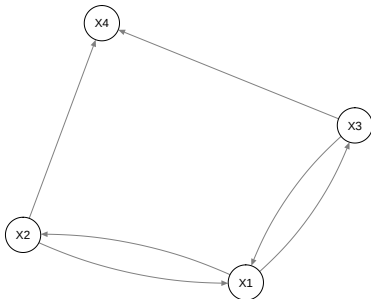
```
set.seed(1); n <- 100
X3 <- runif(n)-0.5
X1 <- X3*X3 + rnorm(n)
X2 <- 1
X4 <- X2+exp(X3+rnorm(n))
X <- cbind(X1,X2,X4)
```

The distribution of  $(X_1, X_3)$  did not change!



```
scm = model2network("[X3] [X1|X3] [X2|X1] [X4|X2:X3]")  
bnlearn::dsep(scm, "X2", "X3", "X1")  
qgraph::qgraph(essentialGraph(amat(scm)))
```

There are two DAGs Markov equivalent to the DAG of this SCM.



In one of them the distribution of  $X_1$  under  $X_2 \leftarrow 1$  changes. Intervening on  $X_3$  allows us to pin down the original DAG.

## More on learning the DAG

If  $X$  is induced by a Structural Causal Model with graph  $G$  then  $X$  satisfies the Markov properties with respect to  $G$ . The model is often strictly smaller than the underlying DAG model, which allows for graph identification.

Some cases when identification is possible:

- In linear models if  $\epsilon$  are non-Gaussian, the DAG can be recovered. Linear non-Gaussian acyclic model (**LiNGAM**).
- Additive noise models:  $X_i = f_i(X_{\text{pa}(i)}) + \epsilon_i$  with nonlinear  $f_i$ , Gaussian  $\epsilon_i$ .

## LiNGAM in action

Package `CompareCausalNetworks` provides a unified interface for the estimation of causal networks that includes a handful of available methods (also LiNGAM).

```
library(CompareCausalNetworks)
# use again the CAD dataset. remove one non-binary variable
data(cad1, package='gRbase')
cad <- data.matrix(cad1)-1;
g.cad <- getParents(cad, method="LINGAM") # learn the DAG
qgraph::qgraph(g.cad)
```

## 5. GMs with latent variables

# What is a hidden variable?

Random vector  $Y = (X, H)$ :  $X$  observed,  $H$  **hidden/latent**.

Observing  $X$ , we only get information about the marginal  $p_X$ .

Since  $p(x) = \sum_h p(h)p(x|h)$ , sometimes  $p(x)$  may be enough to recover  $p(x, h)$ .

## Examples

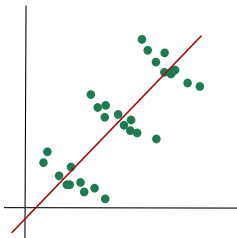
Factor analysis models, hidden Markov models, Markov switching models, mixture models.

## Further Examples

Latent tree models, ancestral graph models, nested Markov models.

## Simpson's paradox

The **Yule-Simpson paradox** says that we may have  $X$  and  $Y$  positively associated but  $X$  and  $Y$  negatively associated conditionally on a third variable  $Z$ .



This together with two examples discusses in Part I shows how important is to observe all variables relevant for the causal analysis.

Often there is a way to tell that such a missing variable exists.

## Fixed graph

If the graph is fixed then, at least in principle, estimation can be performed using the **EM algorithm** (usual disadvantages).

If the graph is decomposable, the M-step is given in closed form.

- packages discussed earlier will apply.

An example of the E-step for the Gaussian case appears on the next slide.

## The $E$ -step for Gaussian distributions

- The  $E$ -step is standard. We work with data of length  $n$  normalized to have mean zero.
- Suppose that  $\Sigma$  represents the **full** covariance matrix estimated at the previous step of the algorithm.
- Let  $S$  be the sample correlation matrix that we are trying to estimate:  
$$S_{XX} = \frac{1}{n} \mathbf{X}^T \mathbf{X}, \quad S_{HX} = \frac{1}{n} \mathbf{H}^T \mathbf{X}, \quad S_{HH} = \frac{1}{n} \mathbf{H}^T \mathbf{H},$$
- Standard formulas:  $\mathbb{E}[H|X] = \Sigma_{HX} \Sigma_{XX}^{-1} X$  and  $\text{var}(H|X) = \Sigma_{HH} - \Sigma_{HX} \Sigma_{XX}^{-1} \Sigma_{XH}$ .
- This gives  $\mathbb{E}[S_{HX}|\mathbf{X}] = \Sigma_{HX} \Sigma_{XX}^{-1} S_{XX}$  and

$$\mathbb{E}[S_{HH}|\mathbf{X}] = \Sigma_{HH} - \Sigma_{HX} \Sigma_{XX}^{-1} \Sigma_{XH} + \Sigma_{HX} \Sigma_{XX}^{-1} S_{XX} \Sigma_{XX}^{-1} \Sigma_{XH}.$$



## Learning the graph structure

Learning the graph structure in the presence of latent variables is hard:

- The number of possible graphs is too large (in fact infinite).
- Estimation for each fixed graph is relatively complicated.
- Models with latent variables fail many regularity assumptions used in standard asymptotic statistics.

Some R packages: `lvnet`, `pcalg` (method FCI), `HMM`.

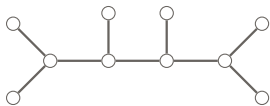
Some cases we cover:

- Latent tree models.
- Latent Gaussian graphical models in high dimensions.
- Briefly: FCI for ancestral graphs.

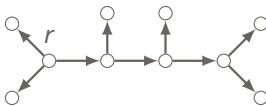
## 5.1 Latent tree models

# Trees

- **Definition:** Tree = undirected graph without cycles
- a tree  $T = (V, E)$ :  $V$  vertices,  $E$  edges

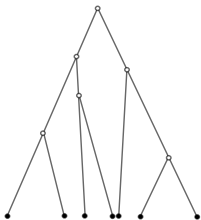


undirected



rooted

- rooted tree often depicted as...
- leaves=degree one nodes
- inner nodes=degree  $\geq 2$  nodes



# Latent tree models

## One step away from tree models

- Graphical models on trees have many nice properties:
  - ▶ exponential families with explicit formulas for the MLE,
  - ▶ dynamic programming for efficient computations of various probabilistic quantities.
- Making some of the variables hidden gives greater flexibility.

## Naive definition of latent tree models

- **Tree-decomposable distribution** a marginal distribution of a tree distribution.
- Discussed by J. Pearl as a natural extension of star-decomposable distributions (mixture model, naive Bayes model, latent class model)

Judea Pearl, *Fusion, Propagation, and Structuring in Belief Networks*, Artificial Intelligence, 1986.

# Motivation

## Statistical motivation

- Applications in:
  - ▶ phylogenetics and linguistics – to model evolutionary processes,
  - ▶ hierarchical clustering,
  - ▶ image processing and network tomography.
- Important concept in causality.
- Many well known statistical models are submodels:
  - ▶ hidden Markov models and naive Bayes models,
  - ▶ phylogenetic models of evolution.

## Gaussian latent tree model

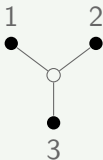
- If  $Y = (X, H)$  is jointly Gaussian, then the marginal distribution of  $X$  is also Gaussian and it satisfies

$$\rho_{ij} = \prod_{e \in \overline{ij}} \rho_e \quad \text{for all } i, j \in [m],$$

where  $\rho_e$  are edge correlations.

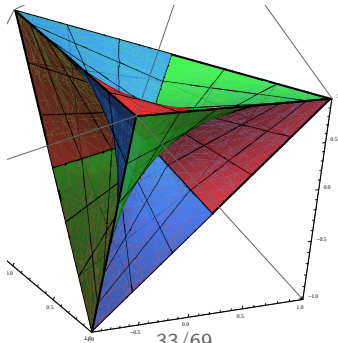
- Moreover, variances  $\sigma_{ii}$  for  $i \in [m]$  are unconstrained.

## Featured on my website



- Consider the tripod tree model.
- $Y = (X_1, X_2, X_3, H)$
- $Y \sim \mathcal{N}_4(0, \Sigma), \Sigma \in N(T)$

- $\rho_{12} = \rho_{h1}\rho_{h2}, \rho_{13} = \rho_{h1}\rho_{h3}, \rho_{23} = \rho_{h2}\rho_{h3}$  and  $\rho_{h1}, \rho_{h2}, \rho_{h3} \in [-1, 1]$ .



## Chow-Liu algorithm

Suppose first we observe all the data. (no hidden variables)

**Problem:** Suppose that we want to find the MLE over the set of **all** tree models for this data.

**Mutual information**  $I(Y_i, Y_j)$  as the Kullback-Leibler divergence between  $p_{ij}$  and the product  $p_i p_j$

$$I_p(Y_i, Y_j) = \sum_{y_i, y_j} p_{ij}(y_i, y_j) \log \frac{p_{ij}(y_i, y_j)}{p_i(y_i) p_j(y_j)}.$$

Moreover,  $I_p(Y_i, Y_j) \geq 0$  and is zero precisely when  $Y_i$  and  $Y_j$  are independent.



For a fixed tree  $T$ :

$$p(y; \hat{\theta}) = \prod_v \hat{p}_v(y_v) \prod_{u-v \in E(T)} \frac{\hat{p}_{uv}(y_u, y_v)}{\hat{p}_u(y_u) \hat{p}_v(y_v)}.$$

The log-likelihood at  $\hat{\theta}$  can be rewritten as

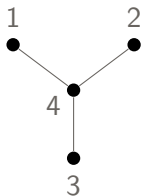
$$n \sum_v \sum_{y_v} \hat{p}_v(y_v) \log \hat{p}_v(y_v) + n \sum_{u-v \in E(T)} I_{\hat{p}}(Y_u, Y_v).$$

**Theorem:** The maximum likelihood tree is the **maximum cost spanning tree** (use **Kruskal's algorithm**).

The same is true in the Gaussian case

- here also:  $I_{\hat{f}}(Y_u, Y_v) = -\frac{1}{2} \log(1 - \hat{\rho}_{uv}^2)$ ,  $\hat{\rho}_{uv}$  = sample correlation

## Exercise: Star tree



- Fixing parameter values,  $\theta_4(1) = 0.6$  and  $\theta_{i|4}$

$$\begin{bmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{bmatrix}, \quad \begin{bmatrix} 0.8 & 0.2 \\ 0.5 & 0.5 \end{bmatrix}, \quad \begin{bmatrix} 0.6 & 0.4 \\ 0.4 & 0.6 \end{bmatrix}$$

we obtain the data generating distribution.

- a simulated matrix of observed mutual informations:

$$\begin{bmatrix} \cdot & 0.000 & 0.003 & 0.043 \\ \cdot & \cdot & 0.004 & 0.027 \\ \cdot & \cdot & \cdot & 0.045 \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix}.$$

- Algorithm: First add edges  $3 - 4$  and  $1 - 4$ . Then,  $2 - 4$ . Since no more edges can be added without introducing cycles, we stop.

```
# define a binary model on the tripod tree
q <- array(0,c(2,2,2,2))
p4 <- c(0.4,0.6)
p14 <- matrix(c(0.7,0.4,0.3,0.6),2,2)
p24 <- matrix(c(0.8,0.5,0.2,0.5),2,2)
p34 <- matrix(c(0.6,0.4,0.4,0.6),2,2)
for (i in 1:2){for (j in 1:2){for (k in 1:2){for (l in 1:2){
  q[i,j,k,l] <- p4[l]*p14[l,i]*p24[l,j]*p34[l,k]
}}}}}
```

```
# generate a sample from this model
n<- 200 #sample size
dat <- sample(1:16,n,replace=TRUE,c(q))
phat <- tabulate(dat)/n
dim(phat) <- c(2,2,2,2)
```

```
#compute mutual informations
MI <- matrix(0,4,4)
for (i in 1:3){
  for (j in (i+1):4){
    pi <- apply(phat,c(i),sum); pj <- apply(phat,c(j),sum)
    pij <- apply(phat,c(i,j),sum)
    MI[i,j]=sum(log(pij/outer(pi,pj))*pij)
  }
}
```

MI

# Structural EM: basic idea

## The goal

- We want to find the maximum likelihood estimator over the union of latent tree models.
- We can assume  $T$  are binary phylogenetic trees.
  - ▶ This can be further refined if needed.

## Exploiting the Chow-Liu algorithm

- If we observed all vertices we could use the Chow-Liu algorithm.
- We use the same idea as in the EM algorithm: first estimate the unobserved mutual informations, then run Chow-Liu on everything.

## Structural EM for Gaussian models

- **Initialize:** Choose a starting binary tree topology  $T^0$  and edge correlations  $\rho^0 = (\rho_e^0)$ . Then, until a convergence criterion is satisfied, perform the two following steps for  $i = 0, 1, \dots$
- **E-step:** Compute expected sample covariance of  $(X, H)$  given the parameters  $T^i, \rho^i$  and the observed vector  $X$ . (c.f. Slide 27)
- **M-step:** Use the Chow-Liu algorithm to update both the tree and edge weights.

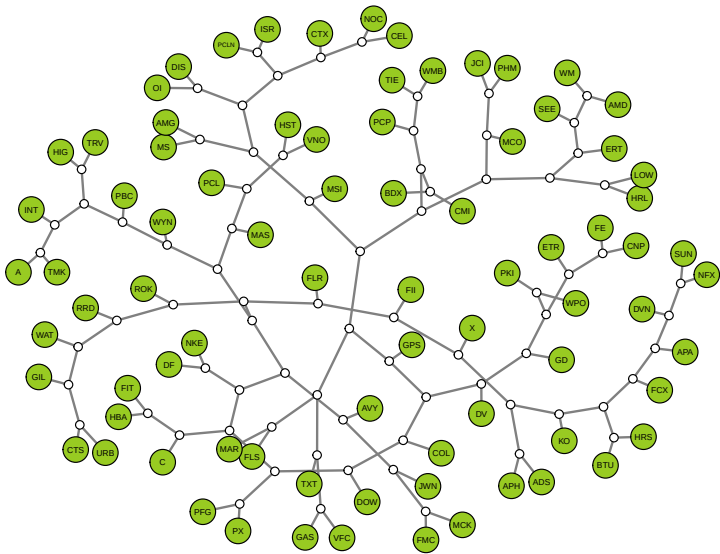
This works subject to some technicalities. . .

Friedman et al., A Structural EM Algorithm for Phylogenetic Inference, Journal of Computational Biology, 2002.

## Structural EM in action

The following code uses the package StructuralEM in the Box.

```
library(huge); data(stockdata)
set.seed(1)
ns <- 80
# subsample 80 stocks from the dataset
sample.stocks <- sample(1:ncol(stockdata$data), ns)
dat = log(stockdata$data[2:1258, sample.stocks] / stockdata$data[1:1257, sample.stocks])
dat <- normalize.data(dat)
# quick heuristic guess for the underlying graph
T0 <- quick.start(dat)
#run the structural EM algorithm
res <- strEM(dat, T0, tol=1e-6)
#output the tree
T <- res$tree;
K <- as_adjacency_matrix(T)
rownames(K) <- colnames(K) <- c(stockdata$info[sample.stocks, 1], rep(NA, ns-2))
qgraph::qgraph(K, color=c(rep("#98CB22", ns), rep("white", ns-2)), vsize=c(rep(3, ns), rep(1, ns-2)))
# to color nodes with economy segments run:
qgraph::qgraph(K, color=c(2*rank(stockdata$info[sample.stocks, 2]), rep("white", ns-2)), vsize=c(rep(3, ns), rep(1, ns-2)))
```



## 5.2 High-dimensional Gaussian graphical models



## Learning the graph in high-dimensions

Let  $Y = (X, H)$  be a **Gaussian** vector with covariance  $\Sigma$ ;  $K = \Sigma^{-1}$ .

### Low-Rank Plus Sparse estimator

Then using standard matrix algebra

$$(\Sigma_{XX})^{-1} = K_{XX} - K_{XH}K_{HH}^{-1}K_{HX}.$$

If the graph of  $Y$  is sparse then  $K$  (and so also  $K_{XX}$ ) is a sparse matrix.

If the size of  $H$  is small compared to  $X$  then  $K_{XH}K_{HH}^{-1}K_{HX}$  has small rank.

Then  $(\Sigma_{XX})^{-1}$  can be written as  $S + L$ , where  $S$  is **sparse** and  $L$  is **low rank**.

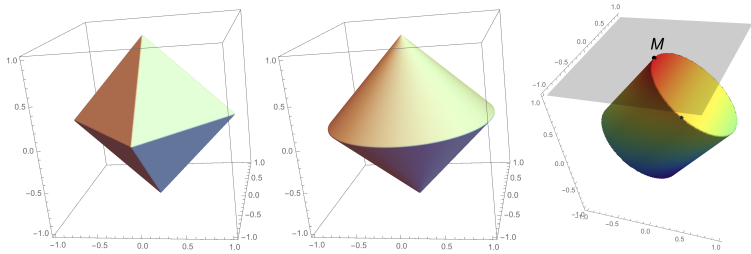
Assuming that  $(\Sigma_{XX})^{-1} = S + L$  where  $S$  is sparse and  $L$  has low rank, we can try to recover  $S$  and  $L$  using convex optimization techniques.

$$\text{maximize} \quad \log \det(S + L) - \text{tr}(W(S + L)) + \lambda \|S\|_1 + \gamma \|L\|_*,$$

where  $W$  is the sample covariance matrix of the observed data,  $\lambda, \gamma$  are positive constants, and  $\|L\|_* = \sum_i \sigma_i(L)$  denotes the **nuclear** norm of  $L$ .

The nuclear norm is a convex envelope of the rank function restricted to matrices with operator norm  $\leq 1$ . (Details in the PhD thesis of Maryam Fazel “Matrix rank minimization with applications” (1435

Google citations))

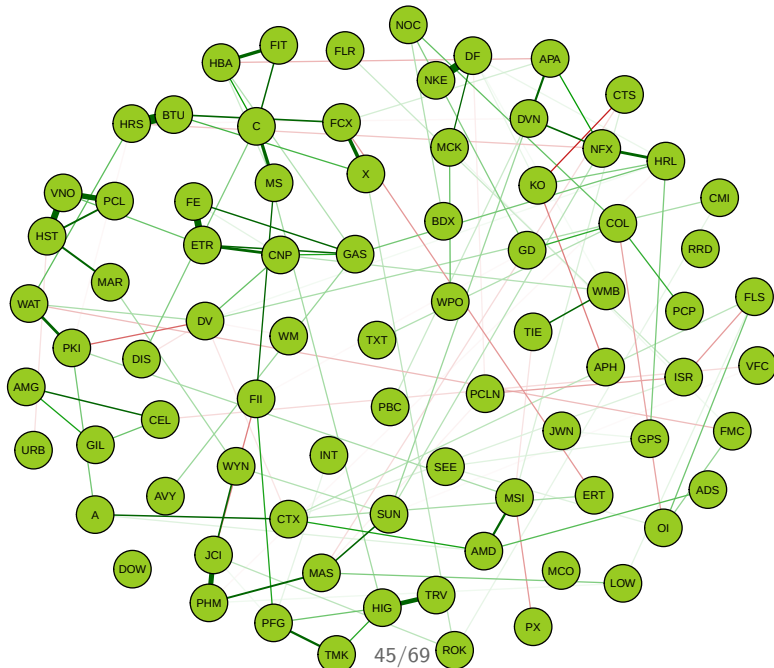


## Using an R implementation

As an example study the stock data discussed earlier (c.f. Slide 40).

```
install.packages("devtools"); library(devtools)
install_github("benjaminfrot/lrpsadmm")
library(lrpsadmm)
gammas <- c(0.1, 0.15, 0.2)
cvpath <- lrpsadmm.cv(dat, gammas = gammas,
                      lambda.ratio = 1e-02, n.lambdas = 30, verbose = TRUE)
best.gamma <- cvpath$best.gamma
best.lambda <- cvpath$best.lambda
best.fit <- cvpath$best.fit$fit
Khat <- -cov2cor(solve(best.fit$S)); diag(Khat) <- 1; rownames(Khat) <- colnames(Khat) <- stockdata$info[
  sample.stocks,1]
Khat <- -cov2cor(best.fit$S); diag(Khat) <- 1; rownames(Khat) <- colnames(Khat) <- stockdata$info[
  sample.stocks,1]
qgraph::qgraph(Khat,color="#98CB22",layout="spring",threshold=1e-3)
# plot according with nodes colored by economy segments
qgraph::qgraph(Khat,color=2*rank(stockdata$info[sample.stocks,2]),layout="spring",threshold=1e-3)
sum(abs(eigen(best.fit$L)$values)>1e-10) # gives that the rank of L is 16.
```

Some extensions to generalized linear models are forthcoming.



## 5.3 Ancestral graphs

## Fix BNs for latent variables

Bayesian networks are not closed under taking margins.

Maximal Ancestral Graphs (MAGs): the smallest family closed under taking margins and containing DAGs.

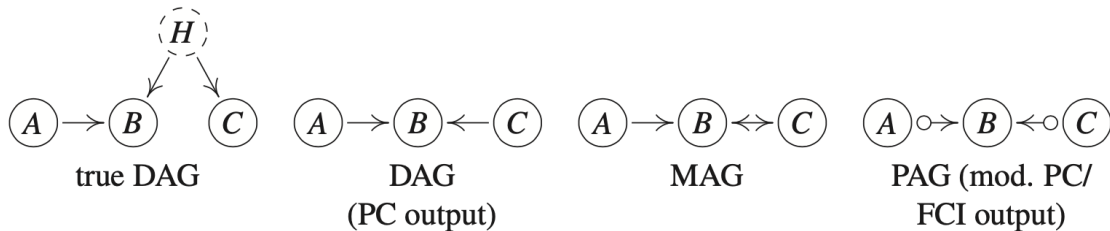
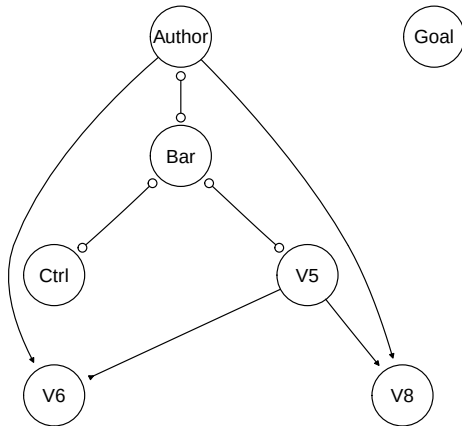
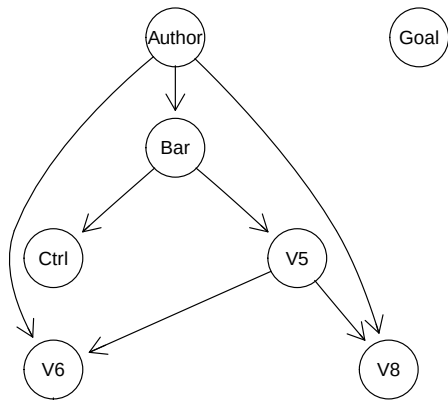


Figure 9.3: Starting with an SCM on the left-hand side, the three graphs on the right encode the set of conditional independences  $(A \perp\!\!\!\perp C)$ . Due to an erroneous causal interpretation, the DAG is not desirable as an output of a causal learning method. In this example, the IPG and the latent projection (ADMG) are equal to the MAG.

## Example ancestral graph

```
library(pcalg)
data("gmG")
p<-ncol(gmG$x)
suffStat<-list(C=cor(gmG$x),n=nrow(gmG$x))
G <- fci(suffStat , indepTest=gaussCitest , labels=nodes(gmG$g), skel.method="stable", alpha=0.01)
```

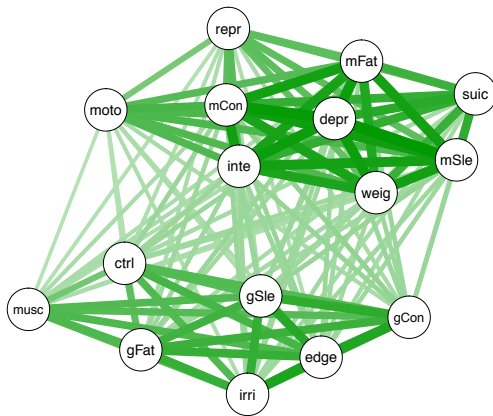


## Part 6: Positive association



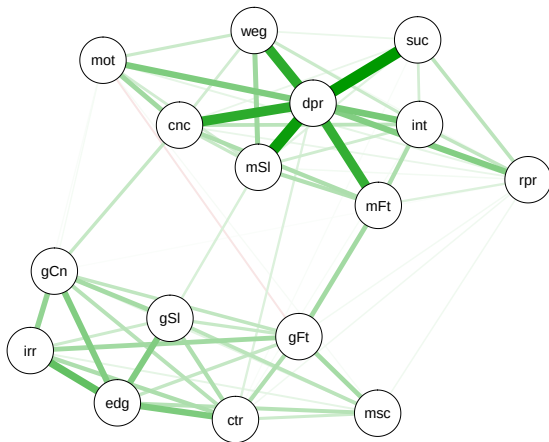
## Example 1: Two psychological disorders

The sample correlations:



Note: All edges are green (positive sample correlations).

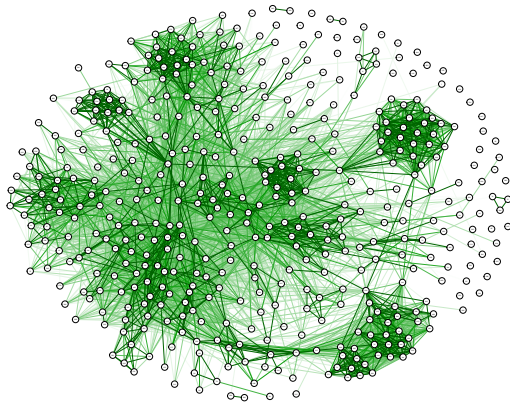
$\ell_1$ -regularized Ising model:



Note: Most edges green (positive cond. log-odds ratios).

## Example 2: S&P 500

graphical lasso estimate of the graph:



Note: All edges green (positive **partial correlations**).

## 6.1 Basic concepts

## Definition

- $X = (X_1, \dots, X_m)$ ,  $\mathcal{X} = \prod_{i=1}^m \mathcal{X}_i \subset \mathbb{R}^m$ , density  $p$ .
- A function  $p$  is **MTP<sub>2</sub>** if:

$$p(x)p(y) \leq p(x \wedge y)p(x \vee y) \quad \text{for all } x, y \in \mathcal{X}.$$

▶ e.g.  $p(1, 1, 0)p(0, 0, 1) \leq p(0, 0, 0)p(1, 1, 1)$

- If  $p > 0$  the condition simplifies.

▶ e.g.  $p(1, 1, 0)p(0, 1, 1) \leq p(0, 1, 0)p(1, 1, 1)$

## Basic properties

- $X = (X_1, \dots, X_m)$  is **positively associated** if for any two *non-decreasing* functions  $\phi, \psi : \mathbb{R}^m \rightarrow \mathbb{R}$

$$\text{corr}\{\phi(X), \psi(X)\} \geq 0.$$

If  $X = (X_1, \dots, X_m)$  is  $\text{MTP}_2$ , then

- (i) any **marginal** distribution is  $\text{MTP}_2$ ;
- (ii) any **conditional** distribution is  $\text{MTP}_2$ ;
- (iii)  $X$  is associated.

S. Karlin, Y. Rinott, *Classes of orderings of measures and related correlation inequalities. I. Multivariate totally positive distributions*, JMVA, 1980.

## Two special cases

**Gaussian model:**  $f(x) = \frac{1}{(2\pi)^{p/2}} \sqrt{\det K} \exp(-x^T K x/2)$

$\text{MTP}_2 \iff K = \Sigma^{-1}$  satisfies  $K_{ij} \leq 0$  for all  $i \neq j$ .

**Binary Ising model:**  $f(x) = \frac{1}{Z(h,J)} \exp(h^T x + x^T J x/2)$

$\text{MTP}_2 \iff J_{ij} \geq 0$  for all  $i \neq j$ .

## 6.3 Modelling



## A restrictive condition?

- $\text{MTP}_2$  constraints appear to be *restrictive*:
  - ▶ 3-dim Gaussian: about 5% of distributions are  $\text{MTP}_2$ ,
  - ▶ 4-dim Gaussian: about 0.09% of distributions are  $\text{MTP}_2$ .
- Less restrictive under additional Markov structure.
- In the 3-dim case
  - ▶ if  $1 \perp\!\!\!\perp 2|3$  then 25% are  $\text{MTP}_2$
  - ▶ if, in addition,  $1 \perp\!\!\!\perp 3|2$  then 50% are  $\text{MTP}_2$
  - ▶ if  $1 \perp\!\!\!\perp 2 \perp\!\!\!\perp 3$  then everything is  $\text{MTP}_2$

Informally: In sparse structures  $\text{MTP}_2$  is more likely.

## Example 3: EPH-gestosis

- Dataset collected 45 years ago in a study on “Pregnancy and Child Development”
- EPH-gestosis (pre-eclampsia): disease **syndrome** for pregnant women; three **symptoms** (high body water retention, high amounts of urinary proteins, elevated blood pressure)
- A syndrome is a set of medical signs and symptoms that are correlated with each other and, often, with a particular disease or disorder.

The sample distribution

$$\begin{bmatrix} \hat{p}_{000} & \hat{p}_{010} & \hat{p}_{001} & \hat{p}_{011} \\ \hat{p}_{100} & \hat{p}_{110} & \hat{p}_{101} & \hat{p}_{111} \end{bmatrix} = \frac{1}{4649} \begin{bmatrix} 3299 & 107 & 1012 & 58 \\ 78 & 11 & 65 & 19 \end{bmatrix}$$

is already  $\text{MTP}_2$ . Equivalently,  $X_1 \perp\!\!\!\perp X_2 \perp\!\!\!\perp X_3 | H$  for a latent binary  $H$ .

## Example 4: Financial time series.

Monthly correlations of global stock markets

$$S = \begin{matrix} & \begin{matrix} Nasdaq & Canada & Europe & UK & Australia \end{matrix} \\ \begin{matrix} Nasdaq \\ Canada \\ Europe \\ UK \\ Australia \end{matrix} & \begin{pmatrix} 1.000 & 0.606 & 0.731 & 0.618 & 0.613 \\ 0.606 & 1.000 & 0.550 & 0.661 & 0.598 \\ 0.731 & 0.550 & 1.000 & 0.644 & 0.569 \\ 0.618 & 0.661 & 0.644 & 1.000 & 0.615 \\ 0.613 & 0.598 & 0.569 & 0.615 & 1.000 \end{pmatrix} \end{matrix}$$

$$S^{-1} = \begin{matrix} & \begin{matrix} Nasdaq & Canada & Europe & UK & Australia \end{matrix} \\ \begin{matrix} Nasdaq \\ Canada \\ Europe \\ UK \\ Australia \end{matrix} & \begin{pmatrix} 2.629 & -0.480 & -1.249 & -0.202 & -0.490 \\ -0.480 & 2.109 & -0.039 & -0.790 & -0.459 \\ -1.249 & -0.039 & 2.491 & -0.675 & -0.213 \\ -0.202 & -0.790 & -0.675 & 2.378 & -0.482 \\ -0.490 & -0.459 & -0.213 & -0.482 & 1.992 \end{pmatrix} \end{matrix}$$

Sampled uniformly this happens with prob.  $< 10^{-6}$ !

## Example 5: Math grades

Data: grades of 88 students in Mechanics, Vectors, Algebra, Analysis, Statistics

$$S = \begin{matrix} & \begin{matrix} \textit{mechanics} & \textit{vectors} & \textit{algebra} & \textit{analysis} & \textit{statistics} \end{matrix} \\ \begin{matrix} \textit{mechanics} \\ \textit{vectors} \\ \textit{algebra} \\ \textit{analysis} \\ \textit{statistics} \end{matrix} & \left( \begin{array}{ccccc} 305.7680 & 127.2226 & 101.5794 & 106.2727 & 117.4049 \\ 127.2226 & 172.8422 & 85.1573 & 94.6729 & 99.0120 \\ 101.5794 & 85.1573 & 112.8860 & 112.1134 & 121.8706 \\ 106.2727 & 94.6729 & 112.1134 & 220.3804 & 155.5355 \\ 117.4049 & 99.0120 & 121.8706 & 155.5355 & 297.7554 \end{array} \right) \end{matrix}$$

$$S^{-1} = \begin{matrix} & \begin{matrix} \textit{mechanics} & \textit{vectors} & \textit{algebra} & \textit{analysis} & \textit{statistics} \end{matrix} \\ \begin{matrix} \textit{mechanics} \\ \textit{vectors} \\ \textit{algebra} \\ \textit{analysis} \\ \textit{statistics} \end{matrix} & \left( \begin{array}{ccccc} 5.2446 & -2.4351 & -2.7395 & 0.0116 & -0.1430 \\ -2.4351 & 10.4268 & -4.7078 & -0.7928 & -0.1660 \\ -2.7395 & -4.7078 & 26.9548 & -7.0486 & -4.7050 \\ 0.0116 & -0.7928 & -7.0486 & 9.8829 & -2.0184 \\ -0.1430 & -0.1660 & -4.7050 & -2.0184 & 6.4501 \end{array} \right) \end{matrix}$$

**Ascent or Phylogeny of the Proboscidea 1926 H.F.O.**

**Species known to Darwin in 1856**

**Genera and Species discovered up to 1926**

**Ascent phyla of Elephants and Mastodons. After Osborn in the year 1926**

$X$  is  $\text{MTP}_2$  in:

- ▶ binary latent class models
- ▶ single factor analysis models

## Recall: Two special cases

**Gaussian model:**  $f(x) = \frac{1}{(2\pi)^{p/2}} \sqrt{\det K} \exp(-x^T K x/2)$

$\text{MTP}_2 \iff K = \Sigma^{-1}$  satisfies  $K_{ij} \leq 0$  for all  $i \neq j$ .

**Binary Ising model:**  $f(x) = \frac{1}{Z(h,J)} \exp(h^T x + x^T J x/2)$

$\text{MTP}_2 \iff J_{ij} \geq 0$  for all  $i \neq j$ .

Convex constraint on the space of the (canonical) parameters with the boundary given by zeros.

Informally: In estimation  $\text{MTP}_2$  leads to sparsity.

## 4.2 Gaussian models and finance

## MLE for Gaussian models

- maximize  $\{\log \det K - \text{tr}(SK)\}$  over  $K \in \text{MTP}_2$ .
- The MLE exists iff there exists  $\Sigma \in \text{PD}_m$  with  $\Sigma \geq S$ .
- It is the unique element  $\hat{K} = \hat{\Sigma}^{-1} \in \text{PD}_m$  that satisfies
  - ▶ **Primal feasibility:**  $\hat{K}_{ij} \leq 0 \quad \forall i \neq j$ ,
  - ▶ **Dual feasibility:**  $\hat{\Sigma}_{ii} = S_{ii} \quad \forall i, \quad \hat{\Sigma}_{ij} \geq S_{ij} \quad \forall i \neq j$
  - ▶ **Complimentary slackness:**  $(\hat{\Sigma}_{ij} - S_{ij}) \hat{K}_{ij} = 0 \quad \forall i \neq j$ .

Note: We get zeros in  $\hat{K}$  for free.



## Example: Optimal Markowitz Portfolio

Global minimum variance portfolio:

minimize  $w^T \Sigma_t^* w$  subject to  $w^T \mathbf{1} = 1$ .

Replacing the unknown true covariance matrix of returns  $\Sigma_t^*$  by some estimator  $\hat{\Sigma}_t$  yields the following analytical solution

$$\hat{w} = \frac{\hat{\Sigma}_t^{-1} \mathbf{1}}{\mathbf{1}^T \hat{\Sigma}_t^{-1} \mathbf{1}}$$

In this setting, estimating  $\Sigma_t^*$  becomes the main problem.

## Covariance matrix estimators

Sample covariance is typically a bad estimator of  $\Sigma_t^*$ .

Structural assumptions give lower variance (higher bias).

- Dynamic factor models: Returns for day  $t$  are given by a linear combination of a (small) collection of latent factors.
- Static factor models: As above but  $\Sigma_t$  does not depend on  $t$ .
- Shrinkage of eigenvalues.
- Regularization of the precision matrix: graphical lasso, CLIME.

## Exploiting total positivity

$\text{MTP}_2$  constraint gives a natural regularizer.

Particularly so when applied to finance:

- **capital asset pricing model (CAPM)** (one factor model with positive loadings) is  $\text{MTP}_2$ .
- latent tree models used for unsupervised learning tasks (e.g. clustering similar stocks).

Data analysis suggests that  $\text{MTP}_2$  regularization performs well where CAPM underfits. It outperforms other methods.

## Extensions to heavy-tailed distributions

Typically the stocks data are log-transformed.

The log-transformed data may still be heavy-tailed.

With small modifications, similar approach works for elliptical distributions, or more generally, for **trans-elliptical distributions**.

**Theorem:** If  $X$  is  $\text{MTP}_2$  and transelliptical (so that  $f(X)$  is elliptical) then  $\Sigma_f^{-1}$  is an M-matrix.

The other direction not true, e.g. **no** t-distribution is  $\text{MTP}_2$ .

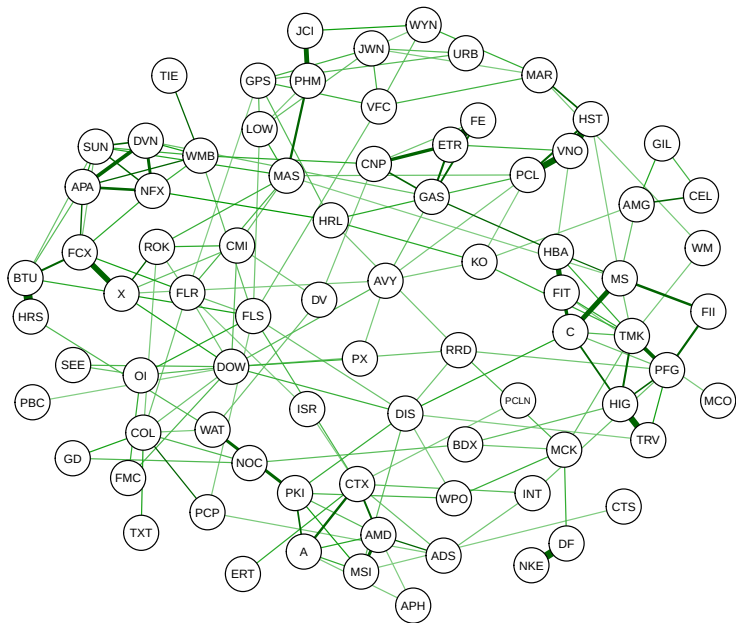
Assuming  $\Sigma_f^{-1}$  is an M-matrix we relax the  $\text{MTP}_2$  assumption.

## Using the golazo package

With the 80 stocks data loaded on Slide 40 do the following.

```
install_github("pzwiernik/golazo")
library(golazo)
R <- cor(dat)
res <- positive.golazo(R,rho=Inf)
K <- res$K
rownames(K) <- colnames(K) <- stockdata$info[sample.stocks,1]
qgraph(-cov2cor(K),layout="spring",threshold=0.05)

#robust version
kR <- cor(dat,method="kendall")
sR <- sin(pi*kR/2)
res <- positive.golazo(sR,rho=Inf)
K <- res$K
rownames(K) <- colnames(K) <- stockdata$info[sample.stocks,1]
qgraph(-cov2cor(K),layout="spring")
```

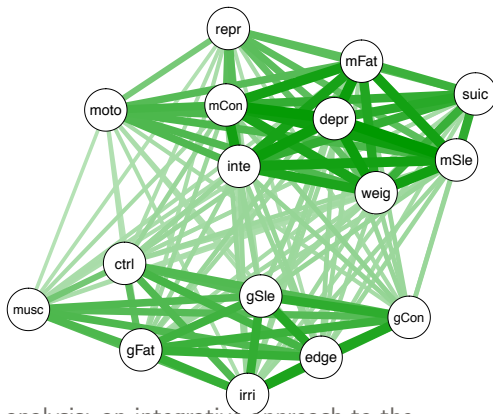


## 4.3 The binary Ising model

## Two psychological disorders, continued

positive sample correlations but not  $MTP_2$

The sample correlation:



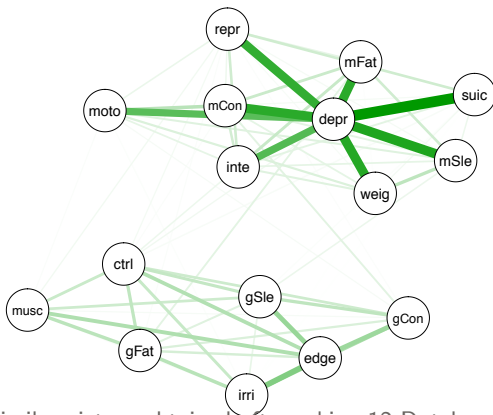
Borsboom and Cramer, Network analysis: an integrative approach to the structure of psychopathology, Annual review of clinical psychology, 9 (2013).



## Two psychological disorders, continued

positive sample correlations but not  $MTP_2$

The  $\hat{J}$  matrix:



Borsboom and Cramer report a similar picture obtained after asking 12 Dutch clinicians for causal relationships!

## Using MTP2binary package

The Gaussian can be estimated using MTP2binary package. The binary Ising model can be estimated using the MTP2binary package.

```
library(MTP2binary)
ncsdata=read.table(file="../DepressionAnxiety.txt") #read in data
colnames(ncsdata)=c("depr", "inte", "weig", "mSle", "moto", "mFat", "repr", "mCon", "suic", "anxi", "
    even", "ctrl", "edge", "gFat", "irri", "gCon", "musc", "gSle") #Define variable names.
XX <- as.matrix(ncsdata)
colnames(XX)=c("depr", "inte", "weig", "mSle", "moto", "mFat", "repr", "mCon", "suic", "anxi", "even", "
    ctrl", "edge", "gFat", "irri", "gCon", "musc", "gSle") #Define variable names.
# remove two variables that are perfectly correlated with each other
X <- 2*(XX[,-(10:11)])-1. # and change to -1/1 coding
n <- nrow(X); d <- ncol(X)
# compute the second moments matrix and the mean vector
M <- t(X)%*%X/n
xbar <- apply(X,2,mean)
p0 <- mtp2.quad.mle(xbar,M,tol=1e-4,maxit=1000) # will take half hour
```

## Other interesting models/packages

`mgm` - Estimation of k-Order time-varying Mixed Graphical Models and mixed VAR(p) models via elastic-net regularized neighborhood regression.

`mixggm` - Mixtures of Gaussian graphical models for model-based clustering with sparse covariance and concentration matrices.

`NetworkChange` - Network changepoint analysis for undirected network data.

`graphicalVAR` - Estimates within and between time point interactions using the Graphical VAR model in combination with regularization.

`mlVAR` - Estimates the multi-level vector autoregression model on time-series data. Three network structures are obtained: temporal networks, contemporaneous networks and between-subjects networks.

`SparseTSCGM` - Computes sparse autoregressive coefficient and precision matrices for time series chain graphical models(TSCGM).

Related articles are on our reading list.